

Project Interim Report
On
School Administration Management
System

RU-100-01-00012
Object Oriented Programming With Java

SESSION: 2025-26



Submitted By
Student name : Shristy Kumari
Reg : No. :2025-03449

**Bachelor of Technology in Computer Science &
Engineering with AI and ML in association with
GOOGLE**

Under the Guidance of
Mr. Ashish Shivhare
**JAVA TRAINER and
EXPERT**

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI , CG

(April 2026)

Abstract

Short summary about your project in 150-200 words. Include the following

- What is your project
- Why is it needed
- What it does

Introduction

Same information about your project but in details you can also add some of the features of your project
Objectives (what your project does) - Manage book records - Track issued books - Reduce manual work - Apply OOP concepts

Scope

This project is suitable for small libraries and can be extended with database integration in future.

System Requirements

Hardware: Computer with 4GB RAM

Software: Java JDK, IDE (VS Code/Eclipse/Edit Plus)

Methodology

Explain how your project works like first we add something, then search, then modify etc.

Flowchart

Includes Flowchart (Start, Menu, Choice, Operations) , and Class Diagram representing system structure.
Sample class diagram –

· **ClassName**

- o Variables inside class
- o Methods inside class
- o If array is present within the class also mention it here

Implementation

Explain different classes used in your program(only explain logic don't paste code here)

Sample Input/Output

Paste screenshot of your terminal with different operations

Future Enhancements

Things you can improve on your project

Implementation

Explain different classes used in your program(only explain logic don't paste code here)

Sample Input/Output

Paste screenshot of your terminal with different operations

Future Enhancements

Things you can improve on your project

Gantt Chart

1. Planning & Structure

A Gantt chart helps you break the project into tasks (design, coding, testing, etc.) and set timelines.

Even if it's already built once, rebuilding or modifying still needs planning.

2. Time Management

It shows:

- When each task starts When it should finish
- This helps avoid delays and last-minute panic.
-

Conclusion

The project successfully demonstrates Java OOP concepts and provides a functional system.

References

Book

Format:

[1] Author(s), *Book Title*, Edition. City: Publisher, Year.

Example:
[1] H. Behltdt, *Java: The Complete Reference*, 11th ed. New York: McGraw-Hill, 2018.

Website

Format:

[3] Author (if any), "Title of page," Website Name. [Online]. Available: URL

Example:

[3] Oracle, "Java Documentation," Oracle. [Online]. Available: <https://docs.oracle.com>

Conference Paper

Format:

[4] Author(s), "Paper title," in Proc. Conference Name, year, pp.

Example:
[4] S. Kumar, "AI in Education," in Proc. IEEE Conf. on AI, 2021, pp. 120-125.

Journal Article

Format:

[2] Author(s), "Title of paper," Journal Name, vol., no., pp., year.

Example:

[2] A. Sharma and R. Gupta, "Machine Learning Techniques," International Journal of AI, vol. 5, no. 2, pp. 45-50, 2020.

School Administration Management System (OOP Project)

Title

Design and Implementation of a School Administration Management System using Inheritance and Polymorphism in Object-Oriented Programming

Description

Educational institutions require structured systems to manage different roles such as teachers and students efficiently. This project focuses on developing a **School Administration Management System** using key **Object-Oriented Programming (OOP)** concepts like **inheritance** and **polymorphism**.

The system will:

- Define a base class **Person** with common attributes
- Extend it into specialized subclasses **Teacher** and **Student**
- Override methods to provide role-specific behavior
- Demonstrate **runtime polymorphism** using a **Person[]** array
- Display role-based information dynamically

This project helps in understanding:

- Code reuse through inheritance
 - Method overriding
 - Dynamic method dispatch (runtime polymorphism)
 - Clean and scalable class design
-

Approach / Steps

Step 1: Create Base Person Class

- Define a class **Person**

- Attributes may include:
 - Name
 - Age
- Define a method:

```
void showRole()
```

- This method will display a general role message
-

Step 2: Create Teacher Class

- Extend `Person` class
- Add specific attributes:
 - Subject
 - Salary
- Override `showRole()` method:

```
@Override
void showRole() {
    System.out.println("I am a Teacher. I teach students.");
}
```

Step 3: Create Student Class

- Extend `Person` class
- Add specific attributes:
 - Roll Number
 - Course
- Override `showRole()` method:

```
@Override
void showRole() {
    System.out.println("I am a Student. I study subjects.");
}
```

Step 4: Demonstrate Polymorphism

- Create an array of type `Person`

Example:

```
Person[] people = new Person[2];
people[0] = new Teacher();
people[1] = new Student();
```

- Iterate through the array and call `showRole()`
-

Step 5: Dynamic Method Dispatch

- The correct method will be invoked at runtime depending on the object type

Example:

```
for (Person p : people) {
    p.showRole();
}
```

Step 6: Testing

- Create objects with sample data
 - Call overridden methods
 - Verify output for both Teacher and Student
-

Expected Learning Outcomes

By completing this project, students will:

- Understand inheritance and class hierarchy
 - Learn method overriding techniques
 - Implement runtime polymorphism
 - Improve modular and reusable code design
 - Apply OOP principles in real-world scenarios
-

Sample Output (Reference)

I am a Teacher. I teach students.
I am a Student. I study subjects.

Submission Requirements

Students must submit:

1. Source Code (Java)
 2. Project Report (Word/PDF)
 3. Viva Presentation
-

Evaluation / Grading Criteria (Total: 25 Marks)

1. Viva (8 Marks)

- Understanding of inheritance → 3 Marks
 - Explanation of polymorphism → 3 Marks
 - Code explanation → 2 Marks
-

2. Report (7 Marks)

- Proper formatting → 2 Marks
 - Explanation of design → 2 Marks
 - Diagrams (Class Diagram / Inheritance Tree) → 2 Marks
 - Clarity → 1 Mark
-

3. Code Implementation (10 Marks)

- Correct inheritance structure → 3 Marks
- Proper method overriding → 3 Marks
- Polymorphism implementation → 2 Marks
- Code readability → 2 Marks